

Digital Payment Anomaly Detection

Ram Kumar Paswan

2026-09-17

1. Executive Summary & Business Problem

In the rapidly expanding digital economy, financial technology companies process millions of micro-transactions daily. A critical operational challenge is **fraud detection**—identifying a microscopic percentage of illegitimate transactions within massive volumes of normal payment activity.

This project aims to build a statistical anomaly detection model to identify fraudulent credit card transactions. We are utilizing a highly imbalanced dataset where fraudulent transactions represent less than 0.2% of the total volume.

2. Data Ingestion & Setup

First, we will load the core statistical libraries and ingest our transaction dataset.

```
# Load necessary libraries
library(dplyr)
library(ggplot2)

# Load the dataset
# Assuming the CSV is in the same folder as this Rmd file
transactions <- read.csv("creditcard.csv")

# Check the dimensions of the data
dim(transactions)
```

```
## [1] 284807    31
```

3. Exploratory Data Analysis: The Imbalance Problem

In anomaly detection, the primary statistical hurdle is class imbalance. Standard predictive models often fail because they simply guess the majority class. Let us quantify and visualize the exact imbalance in our transaction data.

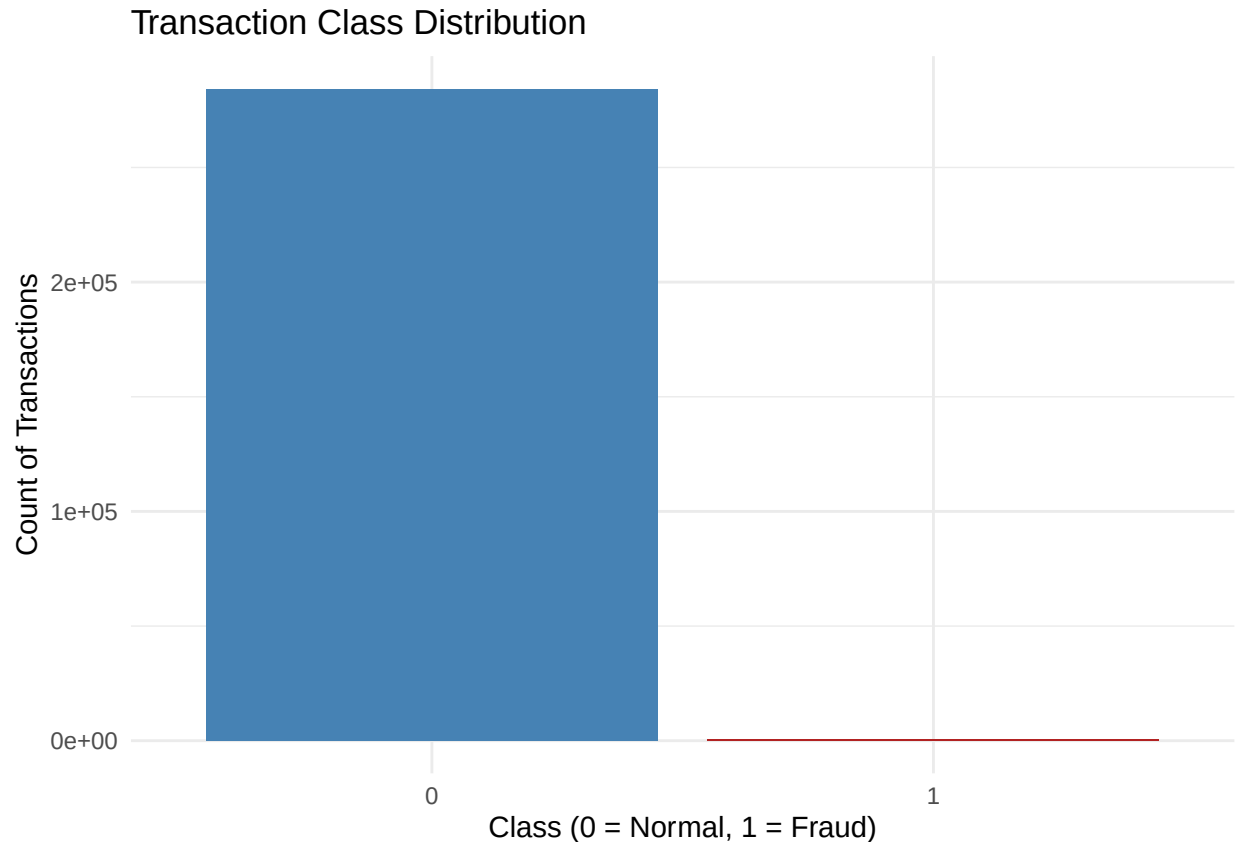
```
# Count normal (0) vs fraudulent (1) transactions
table(transactions$Class)
```

```
##
##      0      1
## 284315   492
```

```
# Calculate the exact percentage of fraud
prop.table(table(transactions$Class)) * 100
```

```
##
##           0           1
## 99.8272514  0.1727486
```

```
# Visualize the imbalance
ggplot(transactions, aes(x = factor(Class), fill = factor(Class))) +
  geom_bar() +
  scale_fill_manual(values = c("0" = "steelblue", "1" = "firebrick")) +
  labs(title = "Transaction Class Distribution",
       x = "Class (0 = Normal, 1 = Fraud)",
       y = "Count of Transactions") +
  theme_minimal() +
  theme(legend.position = "none")
```



4. Model Architecture: Logistic Regression

Because our target variable (Class) is binary (0 or 1), we will deploy a Logistic Regression model. This algorithm calculates the probability that a given transaction is fraudulent based on its features.

To rigorously evaluate our model, we first split the dataset into a training set (80%) and a testing set (20%).

```
# Set a random seed so our results are reproducible
set.seed(42)
```

```
# Create a training (80%) and testing (20%) split
sample_split <- sample(1:nrow(transactions), 0.8 * nrow(transactions))
train_data <- transactions[sample_split, ]
test_data <- transactions[-sample_split, ]
```

```
cat("Training Set Size:", nrow(train_data), "transactions\n")
```

```
## Training Set Size: 227845 transactions
```

```
cat("Testing Set Size:", nrow(test_data), "transactions\n")
```

```
## Testing Set Size: 56962 transactions
```

```
# Train the Logistic Regression Model
# We will use Transaction Time, Amount, and the first three PCA components (V1, V2, V3)
fraud_model <- glm(Class ~ Time + Amount + V1 + V2 + V3,
                   data = train_data,
                   family = "binomial")
```

```
# Display the statistical summary of the model
summary(fraud_model)
```

```
##
## Call:
## glm(formula = Class ~ Time + Amount + V1 + V2 + V3, family = "binomial",
##      data = train_data)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -6.539e+00  1.460e-01 -44.802  < 2e-16 ***
## Time        -1.004e-05  1.293e-06  -7.771  7.80e-15 ***
## Amount      -2.690e-04  1.519e-04  -1.770   0.0767 .
## V1           3.237e-01  3.563e-02   9.084  < 2e-16 ***
## V2           2.729e-01  5.575e-02   4.894  9.86e-07 ***
## V3          -7.420e-01  3.074e-02 -24.140  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 5748.1  on 227844  degrees of freedom
## Residual deviance: 3698.9  on 227839  degrees of freedom
## AIC: 3710.9
##
## Number of Fisher Scoring iterations: 10
```

5. Model Evaluation: The Confusion Matrix

To evaluate our model's real-world viability, we will use it to predict outcomes on our hidden 20% testing set. We calculate the predicted probabilities and classify any transaction with a >50% probability of being fraudulent as a '1'.

```

# Generate probability predictions on the test data
predictions <- predict(fraud_model, newdata = test_data, type = "response")

# Convert probabilities to binary classes (0 or 1) using a 0.5 threshold
predicted_classes <- ifelse(predictions > 0.5, 1, 0)

# Generate the Confusion Matrix
confusion_matrix <- table(Predicted = predicted_classes, Actual = test_data$Class)
print(confusion_matrix)

```

```

##           Actual
## Predicted    0    1
##           0 56848   81
##           1    12   21

```

```

# Calculate base accuracy
accuracy <- sum(diag(confusion_matrix)) / sum(confusion_matrix)
cat("\nOverall Accuracy:", round(accuracy * 100, 2), "%\n")

```

```

##
## Overall Accuracy: 99.84 %

```

6. Advanced Metrics: Precision, Recall, and F1-Score

Because overall accuracy is misleading in highly imbalanced datasets, we must evaluate the model using Precision, Recall, and the F1-Score. In a corporate fraud department, **Recall** is often prioritized to minimize the financial loss from missed fraudulent transactions (False Negatives).

```

# Extract values from the confusion matrix
true_positives <- confusion_matrix["1", "1"]
false_positives <- confusion_matrix["1", "0"]
false_negatives <- confusion_matrix["0", "1"]

# Calculate Metrics
precision <- true_positives / (true_positives + false_positives)
recall <- true_positives / (true_positives + false_negatives)
f1_score <- 2 * ((precision * recall) / (precision + recall))

# Output the results
cat("Precision:", round(precision * 100, 2), "%\n")

```

```

## Precision: 63.64 %

```

```

cat("Recall:", round(recall * 100, 2), "%\n")

```

```

## Recall: 20.59 %

```

```

cat("F1-Score:", round(f1_score, 4), "\n")

```

```

## F1-Score: 0.3111

```

7. Model Optimization: ROC Curve and AUC

To fully evaluate our model's classification capability independent of a strict 0.5 threshold, we plot the Receiver Operating Characteristic (ROC) curve. The Area Under the Curve (AUC) provides a definitive metric of how well the model separates legitimate transactions from fraudulent ones.

```
# Load the ROC library
library(pROC)

# Generate the ROC curve
roc_obj <- roc(test_data$Class, predictions)

# Calculate the Area Under the Curve
auc_value <- auc(roc_obj)
cat("Area Under the Curve (AUC):", round(auc_value, 4), "\n")
```

```
## Area Under the Curve (AUC): 0.8934
```

```
# Plot the ROC curve
plot(roc_obj, col = "darkorange", lwd = 2,
     main = paste("ROC Curve (AUC =", round(auc_value, 4), ")"),
     xlab = "False Positive Rate (1 - Specificity)",
     ylab = "True Positive Rate (Sensitivity)")
```

